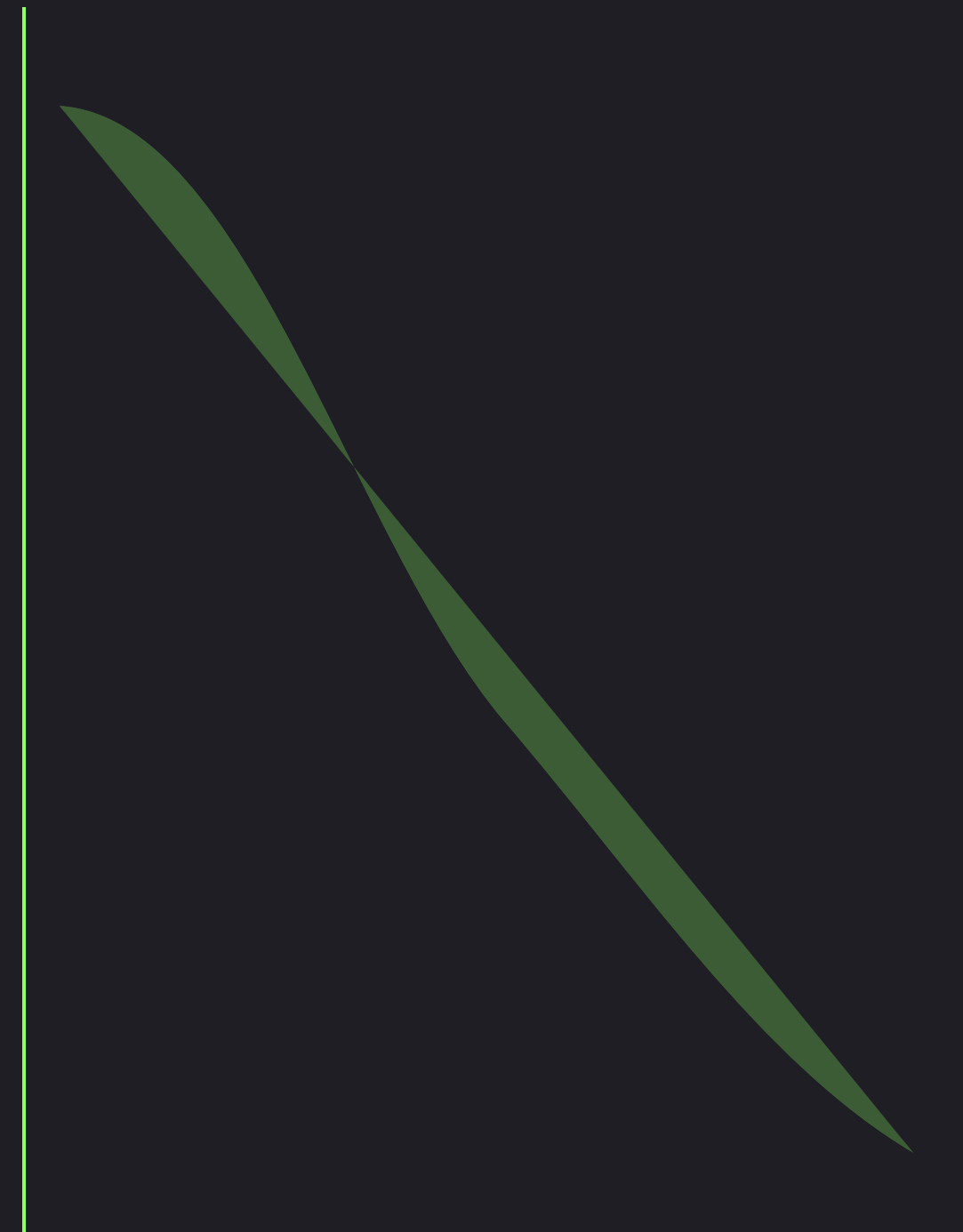


FROM FIRST PRINCIPLES → PRODUCTION

MY GRADIENT DESCENT LEARNING JOURNEY

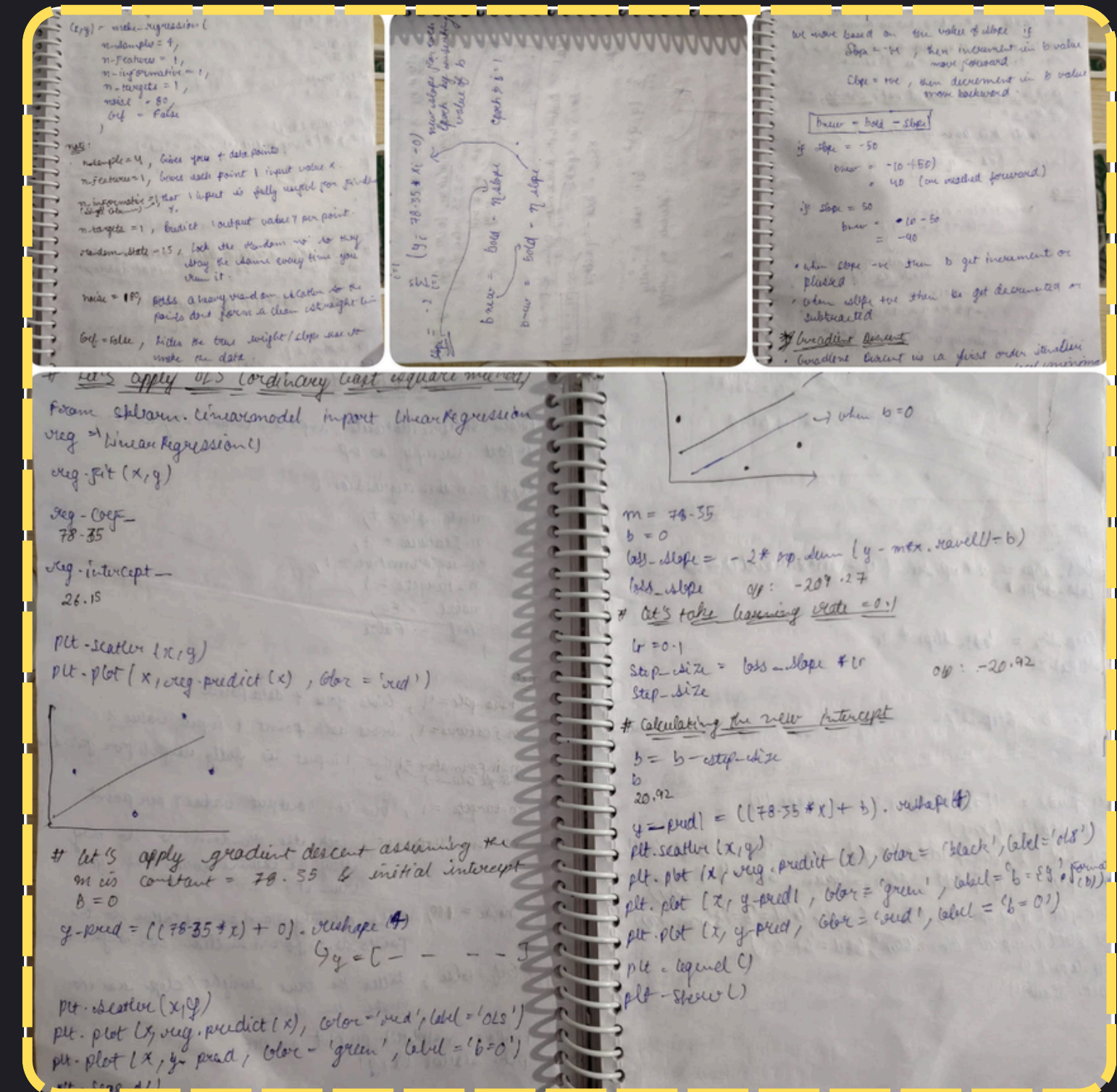
A complete, deep-dive architectural look—from math fundamentals to custom scratch code. A comprehensive glimpse of the optimization landscape I mapped out.

-  clggradadm.ip...
-  credit_card_cu...
-  Gradient_Desc...
-  Gradient_Desc...
-  Gradient_desc...
-  Gradient_Desc...
-  lossfunction.ip...
-  Mini_batch_gr...



SCALING UP: MATH TO MATRIX LOOPS

- Dataset Setup: Built using `make_regression` (`n_samples=4`, `n_features=1`, `noise=80`, `coef=False`) to force a realistic, noisy data distribution.
- OLS Target Benchmark: Ran Scikit-Learn `LinearRegression()` to find the mathematically ideal baseline:
- Intercept (b) = 26.15
- Slope (m) = 78.35
- Custom Iteration Logic: Tested custom optimization by keeping slope constant ($m = 78.35$) and initializing starting intercept at $b = 0$.
- Manual Step Verification:
- $\text{loss_slope} = -2 * \text{np.sum}(y - m * x.\text{ravel}() - b) = -209.27$
- Using Learning Rate (lr) = 0.1
- Step Size = $\text{loss_slope} * lr = -20.92$
- Updated Intercept: $b_{\text{new}} = b - \text{step_size} = 0 - (-20.92) = 20.92$
- Visualizing Convergence: Plotted updates via Matplotlib. The prediction line visibly shifts upwards (from $b=0$ to $b=20.92$), stepping closer to the OLS target benchmark line.



Mathematical Derivatives, Tuning & Batch Variants

- The Vector Gradient: When both intercept (b) and slope (m) vary, we calculate partial derivatives across full vectors:
- $dL/db = -2/n * \sum(y_i - \hat{y}_i)$
- $dL/dm = -2/n * \sum((y_i - \hat{y}_i) * x_i)$
- The Parameter Update Rule: $b_{new} = b_{old} - (\text{learning_rate} * \text{slope})$
- Learning Rate Importance: Tested with initial $b = -10$ and a slope of -50 :
- With Tuning ($lr = 0.01$): $b_{new} = -10 - 0.01 * (-50) = -9.5 \rightarrow$ Gradual, stable step.
- Without Tuning (lr omitted): $b_{new} = -10 - (-50) = 40 \rightarrow$ Unstable, massive overstep.
- The Optimization Spectrum:
- Batch GD:** Computes gradients over all data rows simultaneously. Highly stable but bottlenecks on large datasets.
- Stochastic GD (SGD):** Performs rapid updates using exactly 1 random data point row per step ($idx = np.random.randint()$). Lightning fast but noisy.
- Mini-Batch GD:** Slices data into small, structured matrix sub-batches (e.g., size 35). Combines parallel computing power with high execution speed.
- Production Standard: Scaled custom scratch code into production using Scikit-Learn's optimized library module: `SGDRegressor(learning_rate='constant', eta0=0.01)`.

point. becz this is the direction of steepest descent
that is gradient?

Derivative, like you know before the loss function only depends on the b , that is known as derivative. But if there will be two things, like b and slope then it's known as gradient.

Based on slope it cause drastic change on displacement:

So tackle this learning rate is brought

$$b_{new} = b_{old} - \eta \text{slope}, \quad \eta = 0.01$$

eg:

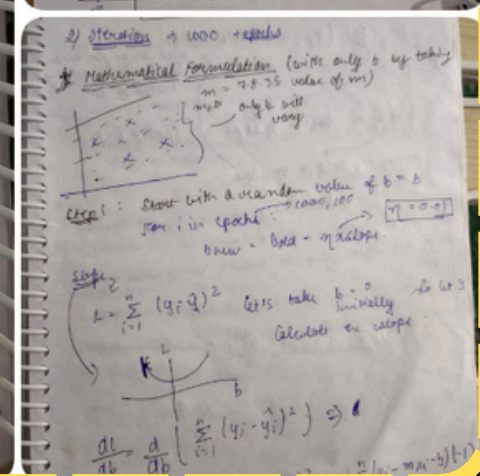
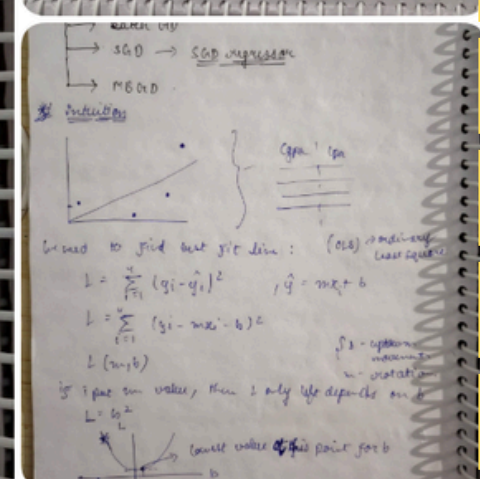
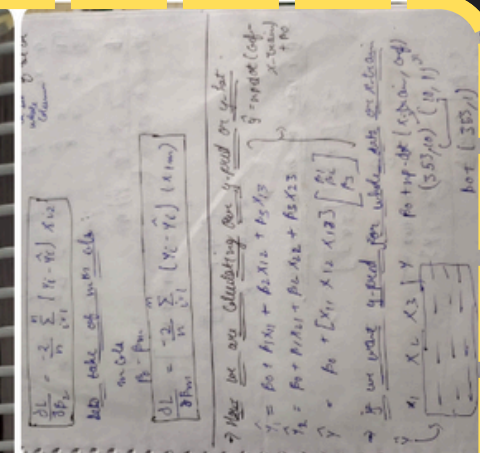
$$b_{new} = -10 - 0.01 \times -50 \\ = -10 + 0.5 = -9.5$$

without η it will reach 40 (drastic change)

$$b_{new} = -10 - 0.01 \times -40 \\ = -9.5 + 0.4 = -9.1$$

$-9.5 \rightarrow -9.1 \rightarrow$ gradually we reach at bottom

? So when to stop in GD



Step 1: Writing Full Batch GD From Scratch

TEST PARAMETERS

$b = -100$

$m = 78.35$

$\alpha = 0.1$

BUILD / FULL BATCH

```
Mini_batch_gradient_descent.ipynb | Stochastic_gradient_descent.ipynb | Gradient_Descent3.ipynb
DP_campus > Gradient_Descent3.ipynb | from sklearn.linear_model import LinearRegression
Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

[13]
... 0.71337703667083

class GDRegressor:
    def __init__(self, learning_rate, epochs):
        self.m = 100,
        self.b = -120
        self.lr = learning_rate
        self.epochs = epochs
    def fit(self, x, y):
        for i in range(self.epochs):
            loss_slope_b = -2 * np.sum(y - self.m * x.ravel() - self.b)
            loss_slope_m = -2 * np.sum((y - self.m * x.ravel() - self.b) * x.ravel())
            self.b = self.b - (self.lr * loss_slope_b)
            self.m = self.m - (self.lr * loss_slope_m)
            print(self.m, self.b)
    def predict(self, x):
        return self.m * x + self.b

[14]

gd = GDRegressor(0.001, 50)

[15]

gd.fit(x_train, y_train)

[16]
... [28.72415564] -2.095758658639012

y_pred = gd.predict(x_test)
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)

[17]
... 0.7129234154065953
```

Inside the fit(self, x, y) Mathematical Architecture

```
loss_slope_b = -2 * np.sum(
    y - self.m * x.ravel() - self.b)

loss_slope_m = -2 * np.sum((
    y - self.m * x.ravel() - self.b) * x.ravel())

self.b -= a * loss_slope_b
self.m -= a * loss_slope_m
```

01 PREDICT

02 MEASURE ERROR

03 UPDATE

MATH LOOP / UPDATE

```
Mini_batch_gradient_descent.ipynb Stochastic_gradient_descent.ipynb Gradient_Descent3.ipynb
DP_campus > Gradient_Descent3.ipynb > class GDRegressor:
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
0.71337703667083

class GDRegressor:
    def __init__(self, learning_rate, epochs):
        self.m=100,
        self.b=-120
        self.lr=learning_rate
        self.epochs=epochs
    def fit(self, x, y):
        for i in range(self.epochs):
            loss_slope_b=-2 * np.sum(y-self.m*x.ravel()-self.b)
            loss_slope_m=-2*np.sum((y-self.m*x.ravel()-self.b)*x.ravel())
            self.b=self.b-(self.lr*loss_slope_b)
            self.m=self.m-(self.lr*loss_slope_m)
            print(self.m, self.b)
    def predict(self, x):
        return self.m*x+self.b

[14]

gd=GDRegressor(0.001, 50)

[15]

gd.fit(x_train, y_train)

[16]

[28.72415564] -2.095758658639012

y_pred=gd.predict(x_test)
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)

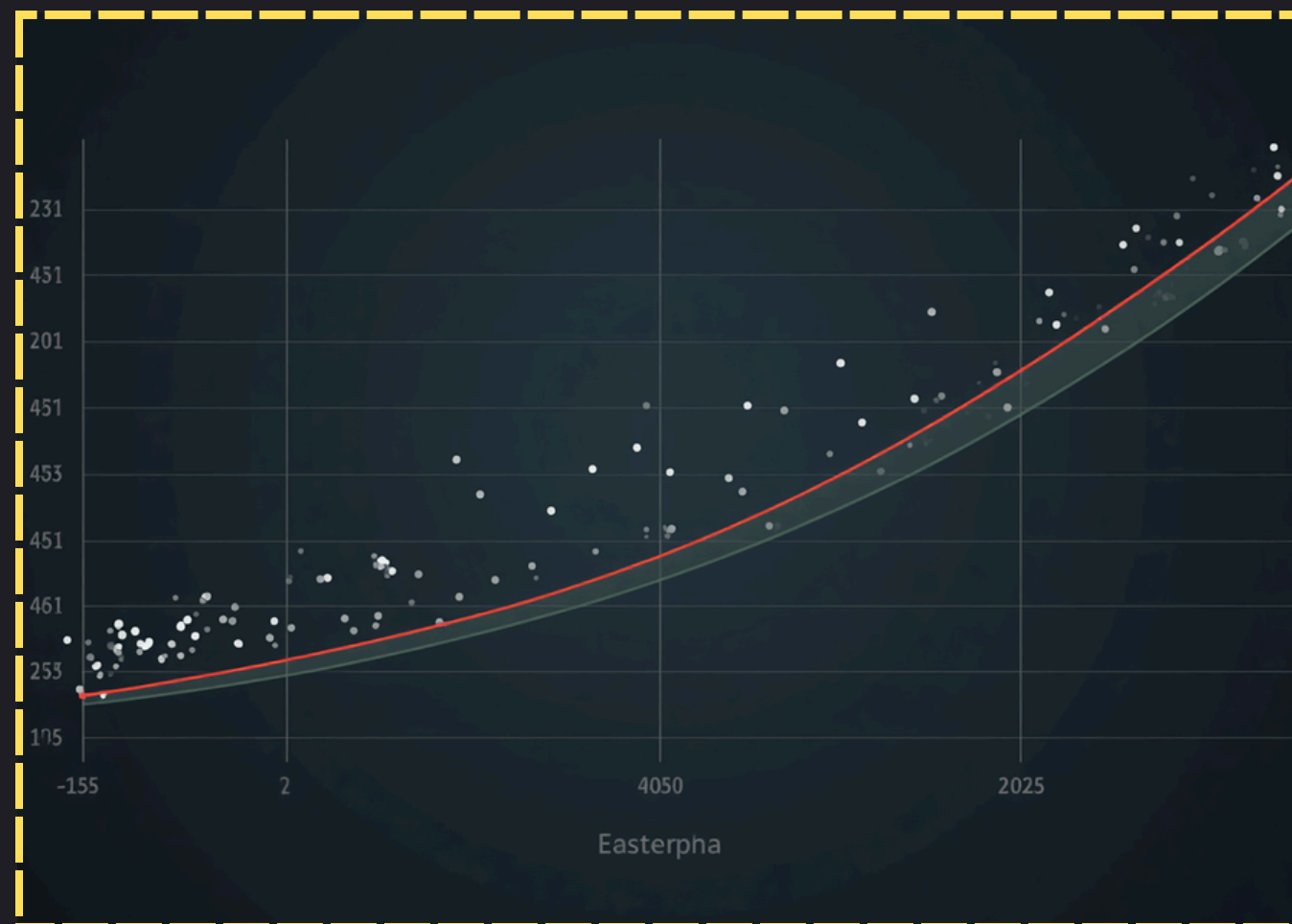
[17]

0.7129234154065953
```

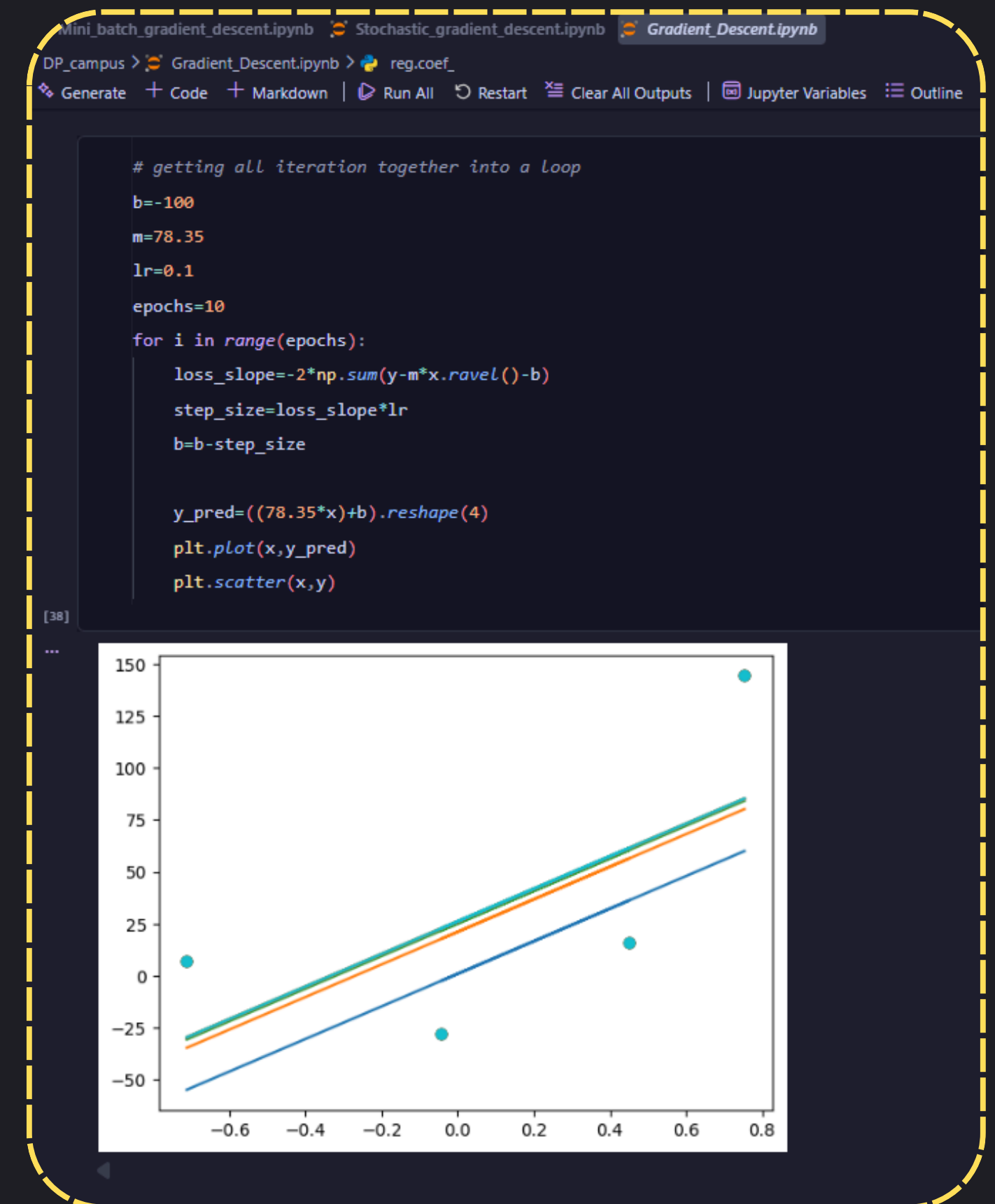

Watching the Algorithm Learn Frame-by-Frame

Each full-data update shifts the prediction line. Over 10 epochs, the fit moves smoothly toward the center of the data.

ERROR ↓



CONVERGENCE / 10 EPOCHS



The Compute Wall of Big Data

Batch GD recalculates the gradient across the entire dataset for every update. Accurate—but expensive at scale.



SGD: ONE RANDOM POINT / UPDATE

SCALE / BOTTLENECK

```
Mini_batch_gradient_descent.ipynb Stochastic_gradient_descent.ipynb Gradient_Descent3.ipynb
DP_campus > Gradient_Descent3.ipynb > class GDRegressor:
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
0.71337/03667083

class GDRegressor:
    def __init__(self, learning_rate, epochs):
        self.m=100,
        self.b=-120
        self.lr=learning_rate
        self.epochs=epochs
    def fit(self, x, y):
        for i in range(self.epochs):
            loss_slope_b=-2 * np.sum(y-self.m*x.ravel()-self.b)
            loss_slope_m=-2*np.sum((y-self.m*x.ravel()-self.b)*x.ravel())
            self.b=self.b-(self.lr*loss_slope_b)
            self.m=self.m-(self.lr*loss_slope_m)
            print(self.m, self.b)
    def predict(self, x):
        return self.m*x+self.b

[14]

gd=GDRegressor(0.001, 50)

[15]

gd.fit(x_train, y_train)

[16]

... [28.72415564] -2.095758658639012

y_pred=gd.predict(x_test)
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)

[17]

... 0.7129234154065953
```

Coding Class SGDRegressor From Scratch

```
i = np.random.randint(0, len(x))
y_hat = self.m * x[i] + self.b

b_grad = -2 * (y[i] - y_hat)
m_grad = -2 * (y[i] - y_hat) * x[i]

self.b -= a * b_grad
self.m -= a * m_grad
```

**FAST UPDATES
NOISIER PATH**

r^2 _score → validate

BUILD / STOCHASTIC

```
class SGDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs # typo mapping epoch argument to self.epochs

    def fit(self, x_train, y_train):
        # init your coefs
        self.intercept_ = 0
        # Only initialize weights for columns/features, not rows
        self.coef_ = np.ones(x_train.shape[1])

        for i in range(self.epochs):
            for j in range(x_train.shape[0]):
                # Pull 1 random data point row index
                idx = np.random.randint(0, x_train.shape[0])

                # Predict: y_hat = mx + b
                y_hat = np.dot(x_train[idx], self.coef_) + self.intercept_

                # Calculate Intercept (b) Derivative
                intercept_der = -2 * (y_train[idx] - y_hat)
                self.intercept_ = self.intercept_ - (self.lr * intercept_der)

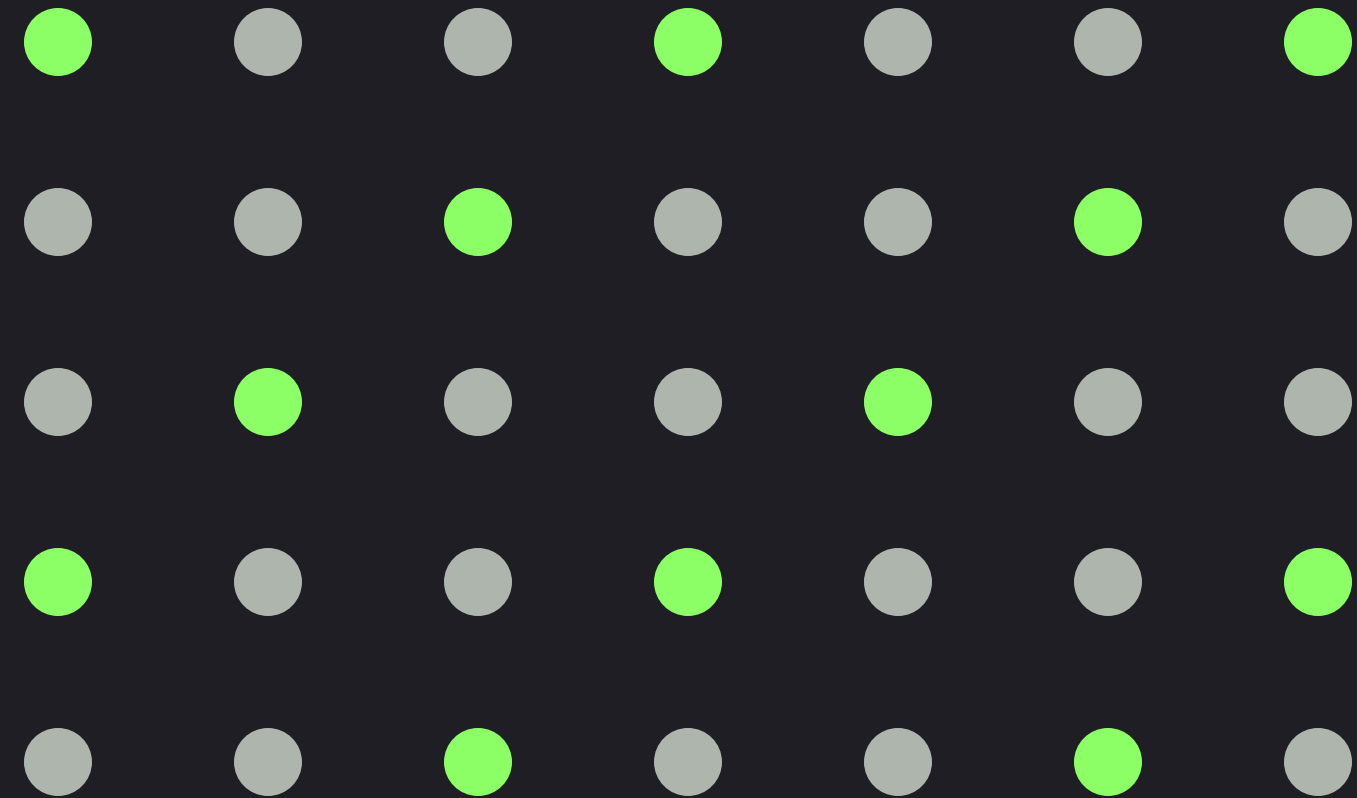
                # Calculate Coefficient (m) Derivative
                # FIX 3: Simplified scalar-vector multiplication
                coef_der = -2 * (y_train[idx] - y_hat) * x_train[idx]
                self.coef_ = self.coef_ - (self.lr * coef_der)

    def predict(self, x_test):
        return np.dot(x_test, self.coef_) + self.intercept_
```


Balancing Speed and Stability

Mini-batch GD groups small subsets for parallel computation—faster than full batch, steadier than SGD.

```
batch_size = 35
```



Implementing Class MBGDRegressor

```
idx = np.random.choice(n, batch_size)
Xb, yb = X[idx], y[idx]

y_hat = Xb @ self.m + self.b
error = yb - y_hat

b_grad = (-2 / batch_size) * error.sum()
m_grad = (-2 / batch_size) * Xb.T @ error
```

SAMPLE → VECTORIZE → SCALE → UPDATE

```
Mini_batch_gradient_descent.ipynb
DP_campus > Mini_batch_gradient_descent.ipynb > Mini batch gradient using sgd regressor > sgd=SGDRegressor(learning_rate='constant',eta0=0.1)
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

class MBGDRegressor:
    def __init__(self, batch_size=10, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size

    def fit(self, x_train, y_train):
        # Initialize coefficients
        self.intercept_ = 0
        self.coef_ = np.ones(x_train.shape[1])

        for i in range(self.epochs):
            for j in range(int(x_train.shape[0] / self.batch_size)):
                # 1. Sample mini-batch indices
                idx = random.sample(range(x_train.shape[0]), self.batch_size)

                # 2. Predict on the mini-batch
                y_hat = np.dot(x_train[idx], self.coef_) + self.intercept_

                # 3. Calculate derivatives
                intercept_der = -2 * np.mean(y_train[idx] - y_hat)
                # FIXED: Removed the extra trailing matrix from np.dot and added proper batch scaling
                coef_der = -2 * np.dot(x_train[idx].T, (y_train[idx] - y_hat)) / self.batch_size

                # 4. Update the intercept and coefficients
                self.intercept_ = self.intercept_ - (self.lr * intercept_der)
                self.coef_ = self.coef_ - (self.lr * coef_der)

            print("Final Intercept:", self.intercept_)
            print("Final Coefficients:", self.coef_)

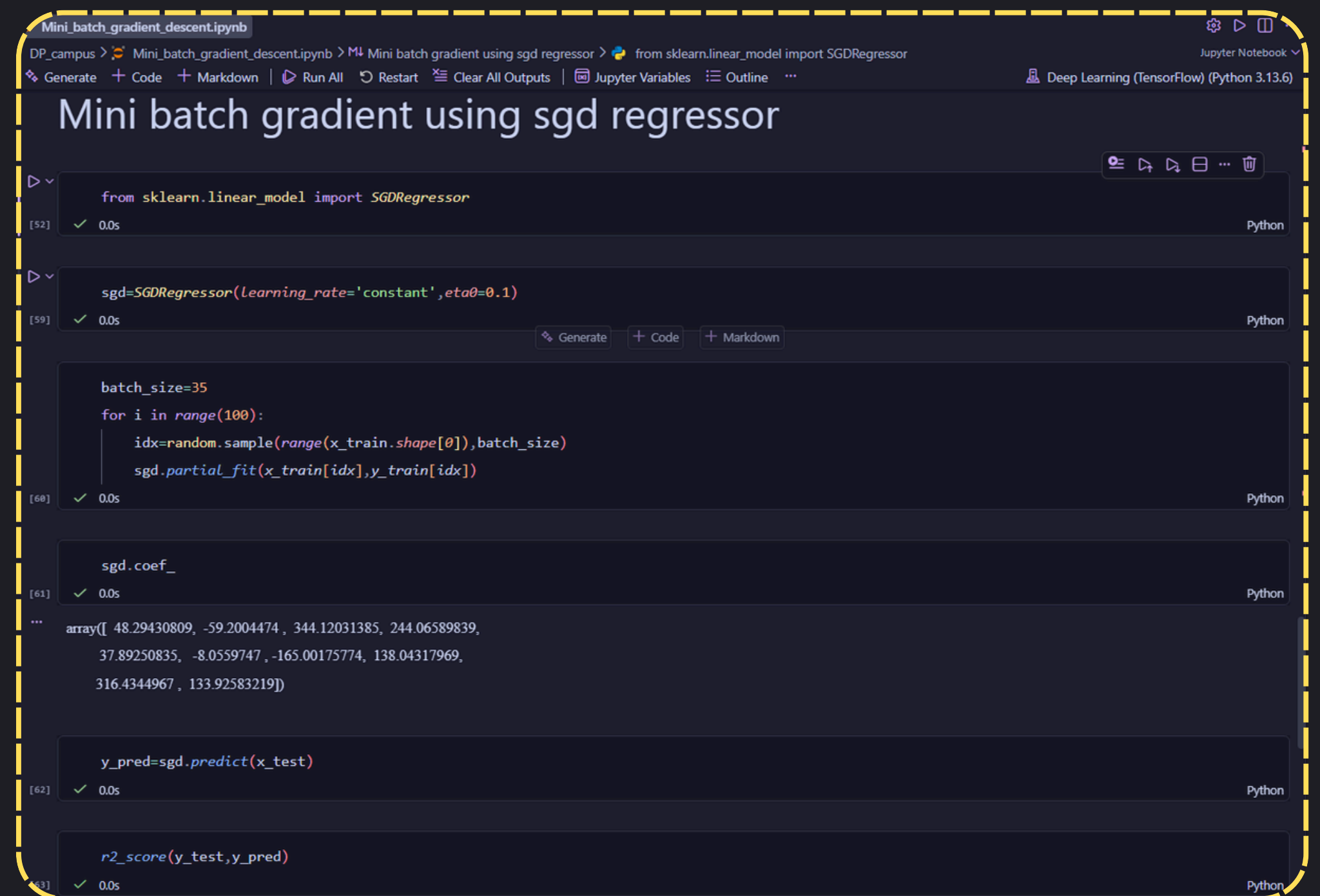
    def predict(self, x_test):
        return np.dot(x_test, self.coef_) + self.intercept_

[46] ✓ 0.0s
```

Scaling Up to Industry Production Standards

Move from scratch logic to optimized, tested library primitives—without losing the mental model.

```
SGDRegressor(  
    learning_rate='constant',  
    eta0=0.1  
)  
model.partial_fit(X_batch, y_batch)
```



```
Mini_batch_gradient_descent.ipynb  
DP_campus > Mini_batch_gradient_descent.ipynb > Mini batch gradient using sgd regressor > from sklearn.linear_model import SGDRegressor  
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline ... Deep Learning (TensorFlow) (Python 3.13.6)  
Jupyter Notebook  
Mini batch gradient using sgd regressor  
from sklearn.linear_model import SGDRegressor  
sgd=SGDRegressor(learning_rate='constant',eta0=0.1)  
batch_size=35  
for i in range(100):  
    idx=random.sample(range(x_train.shape[0]),batch_size)  
    sgd.partial_fit(x_train[idx],y_train[idx])  
sgd.coef_  
array([[ 48.29430809, -59.2004474, 344.12031385, 244.06589839,  
        37.89250835, -8.0559747, -165.00175774, 138.04317969,  
        316.4344967, 133.92583219])  
y_pred=sgd.predict(x_test)  
r2_score(y_test,y_pred)
```


Finding the Sweet Spot for Stability

TOO HOT

$$\alpha = 0.1$$

Bouncing / divergence

CONTROLLED

$$\eta_0 = 0.01$$

max_iter = 100

Validate with r^2 _score

THE MODEL IMPROVES. SO DO WE.

The Optimization Journey Never Stops 🚀

Master the scratch logic. Understand the matrix math. Then scale with high-performance libraries.

CUSTOM MATRIX CALCULATIONS ↔ SCIKIT-LEARN

💬 Drop your optimization questions below
and save this guide!

KEEP ITERATING

